SIMATS ENGINEERING

Assignment 2

COURSE NAME Data Structures COURSE CODE CSA0304

Register Number 192572083A

Name NISHA S

Assignment Questions

1. Explain the role of a stack in converting an infix expression to postfix.

2. Convert A + B * C - D to postfix and show stack contents after each token is processed.

Question 1. Role of a Stack in Infix to Postfix Conversion

1.1 Introduction

An infix expression places operators between operands, such as A + B. A postfix expression places operators after their operands, such as AB+. Postfix notation needs no parentheses and can be evaluated directly using a stack.

A stack follows the Last-In, First-Out (LIFO) principle. During infix-to-postfix conversion, a stack temporarily stores operators and opening parentheses while operands are added directly to the output.

1.2 Why the stack is needed

Operator precedence: Multiplication and division have higher precedence than addition and subtraction. The stack delays lower-priority operators until higher-priority operations are processed.

Associativity: For left-associative operators (+, -, *, /), existing operators of equal precedence are popped before a new operator is pushed.

Parentheses: An opening parenthesis is pushed onto the stack. When a closing parenthesis appears, operators are popped until the matching opening parenthesis is removed.

1.3 Conversion algorithm

1. Create an empty operator stack and an empty postfix output.

2. Read the infix expression from left to right, one token at a time.

3. If the token is an operand, append it directly to the postfix output.

4. If the token is an opening parenthesis, push it onto the stack.

5. If the token is a closing parenthesis, pop operators into the output until the matching opening parenthesis is found; then discard that parenthesis.

6. If the token is an operator, pop any operator of higher precedence (or equal precedence when the incoming operator is left-associative), then push the incoming operator.

7. After all tokens have been read, pop the remaining operators into the output.

1.4 Precedence rules

Operator	Precedence	Associativity
* /	Higher	Left to right
+ -	Lower	Left to right

Question 2. Convert A + B * C - D into Postfix

2.1 Given expression

Infix: A + B * C - D

Since multiplication has higher precedence than addition and subtraction, B * C is performed first. Addition and subtraction have equal precedence and are evaluated from left to right.

2.2 Stack trace after each token

The stack is displayed from bottom to top (left to right). A dash indicates an empty stack. The final row shows the operators remaining after the last input token.

Step	Token	Action	Stack	Postfix output
1	A	Operand: add to output	-	A
2	+	Push +	+	A
3	B	Operand: add to output	+	AB
4	*	Higher precedence: push *	+ *	AB
5	C	Operand: add to output	+ *	ABC
6	-	Pop * and +, then push -	-	ABC*+
7	D	Operand: add to output	-	ABC*+D
8	End	Pop remaining -	-	ABC*+D-

2.3 Step-by-step explanation

Step 1: A is an operand; place it in the output. Step 2: Push + onto the stack. Step 3: Append B to the output.

Step 4: Push * because it has higher precedence than +. Step 5: Append C to the output.

Step 6: On reading -, pop * and then + because they have higher or equal precedence. Append both to the output and push -. Step 7: Append D. Step 8: Pop the final - operator.

2.4 Final answer

Infix expression: A + B * C - D

Postfix expression: ABC*+D-

Verification: The postfix expression corresponds to (A + (B * C)) - D, which preserves the precedence and left-to-right associativity of the original infix expression.

Conclusion

The stack stores operators temporarily so that precedence, associativity, and parentheses are handled correctly. For the given expression, the converted postfix notation is ABC*+D-.